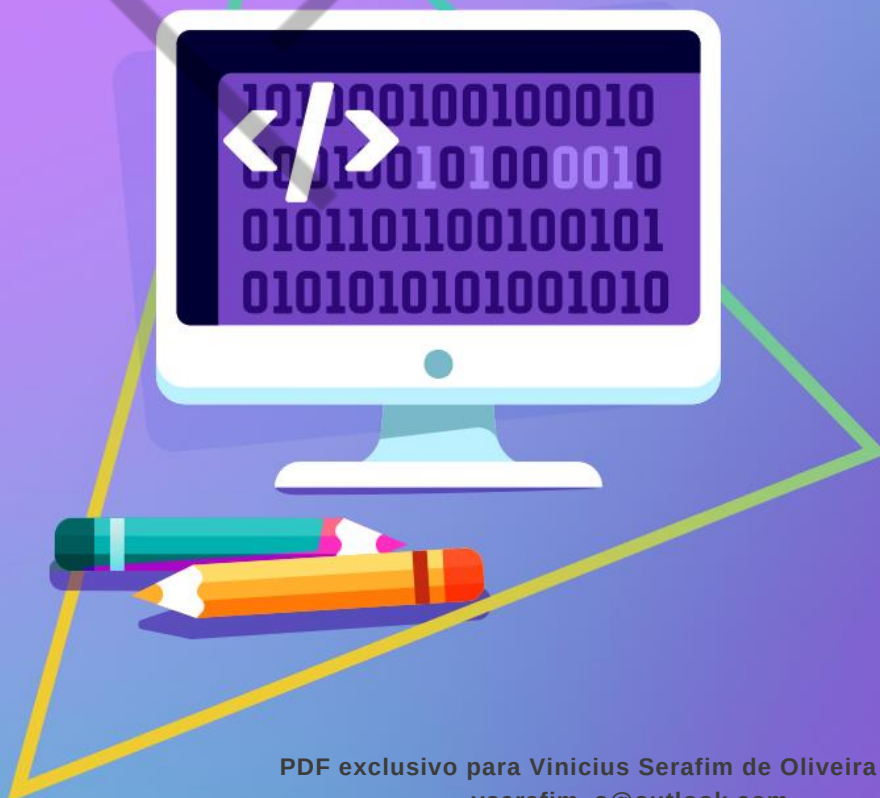


HACKER ATTACK

HORA DE "CODAR"



6

LISTA DE FIGURAS

Figura 1 – Processo de compilação	6
Figura 2 – Processo de interpretação	6
Figura 3 – Processo misto de compilação e interpretação	8
Figura 4 – Teste do método calcEnergy.....	19
Figura 5 –Teste do método calcWater	19
Figura 6 – Teste injetando comando	19
Figura 7 – Eval() com escopo reduzido	21
Figura 8 – Sem eval() no código	22

EXEMPLO

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo de <i>query</i> dinâmica em Java	10
Código-fonte 2 – Sintaxe do método eval()	11
Código-fonte 3 – Exemplo utilizando o método eval()	12
Código-fonte 4 – Exemplo de código malicioso com o método eval()	12
Código-fonte 5 – Exemplo básico de metaprogramação em Python.....	13
Código-fonte 6 – Exemplo de <i>query</i> dinâmica em Java com três parâmetros	15
Código-fonte 7 – Exemplo de código Java que lista os métodos de uma classe	16
Código-fonte 8 – Código Python do <i>Smart Meter</i>	18
Código-fonte 9 – Código Python com <i>locals</i> no <i>Smart Meter</i>	20
Código-fonte 10 – Código Python sem eval() no <i>Smart Meter</i>	22

SUMÁRIO

PROGRAMAÇÃO DINÂMICA	5
1 INTRODUÇÃO	5
2 COMPILAÇÃO	5
3 INTERPRETAÇÃO	6
3.1 Abordagem mista	7
3.1.1 Mista (compilação + interpretação)	7
4 CONCEITO DE PROGRAMAÇÃO DINÂMICA	9
5 PROGRAMAÇÃO DINÂMICA EM PYTHON	11
5.1 Método eval()	11
5.2 Técnicas de metaprogramação em Python	13
5.2.1 O conceito de metaprogramação	13
6 PROGRAMAÇÃO DINÂMICA EM JAVA	14
6.1 Geração de queries	15
6.2 Técnicas de metaprogramação em Java	16
7 LABORATÓRIO – ANÁLISE DE VULNERABILIDADES	17
7.1 Redução de escopo do eval()	20
7.2 Remoção do eval()	21
CONCLUSÃO	22
REFERÊNCIAS	24

PROGRAMAÇÃO DINÂMICA

1 INTRODUÇÃO

Neste capítulo, trabalharemos em um tema correlacionado e dentro da área de programação, envolvendo o assunto de Programação Dinâmica, uma técnica que permite ao desenvolvedor projetar código que possui flexibilidade para lidar com as diferentes informações provenientes do usuário ou de um banco de dados.

A Programação Dinâmica permite que o código seja gerado dinamicamente, ou seja, enquanto o programa estiver em execução (em *runtime*), de forma que o código possa ser alterado sem a necessidade de uma recompilação do código.

Mas antes de estudarmos especificamente essa técnica, é necessário conhecer um dos aspectos principais da arquitetura de uma linguagem de programação: a maneira como é gerado o código executável.

Existem três formas utilizadas atualmente para a geração do código executável:

- Compilação.
- Interpretação.
- Mista (Compilação + Interpretação).

2 COMPILAÇÃO

O processo de compilação envolve a tradução de todo o código-fonte (programa fonte) diretamente para código executável (programa objeto). Por se tratar de uma conversão definitiva, o processo não permite flexibilidade na execução do código, necessitando realizar uma nova compilação no caso de modificações no código-fonte.

Esse processo é muito utilizado nas linguagens antigas de programação, pois apesar de a compilação consumir um tempo elevado para a geração do código executável, este possui grande performance no momento da execução.

Vale lembrar de uma restrição nessa modalidade de execução, o código executável depende da arquitetura do sistema, pois as otimizações aplicadas são orientadas à plataforma específica para a qual o código foi compilado.



Figura 1 – Processo de compilação
Fonte: DELGADO (2018)

Na Figura “Processo de compilação”, é possível observar que o processo de tradução é unidirecional e não permite adaptações, uma vez gerado o programa objeto.

3 INTERPRETAÇÃO

A interpretação de código realiza um processo bem mais dinâmico do que a compilação, vista na subseção anterior.

O interpretador possui como função principal realizar a tradução do código-fonte conforme necessidade (em tempo real), o que permite a utilização de ferramentas de depuração (como ferramentas de *debugging*) e de modificação dinâmica do código sem a necessidade de recompilar totalmente o código.



Figura 2 – Processo de interpretação
Fonte: DELGADO (2018)

A Figura “Processo de interpretação” descreve o processo de interpretação de um código, note que o processo não apenas envolve execução sob demanda de código, como também permite adaptar a tradução conforme os dados de entrada, tornando possível o ajuste do código de acordo com a massa de dados.

O uso de um interpretador acaba resultando em uma baixa performance de execução pelo fato de traduzir *on-the-fly* o código-fonte, mas, em contrapartida, oferece a possibilidade de montagem dinâmica do código de uma aplicação, sem a necessidade de construir o código executável de todo o sistema a cada modificação.

3.1 Abordagem mista

3.1.1 Mista (compilação + interpretação)

A abordagem mista faz uma mescla das duas formas anteriores, de modo a ter um desempenho intermediário de tradução e de execução.

O Java é o maior exemplo de linguagem de programação que segue esse formato de arquitetura, ilustrado na Figura “Processo misto de compilação e interpretação”.

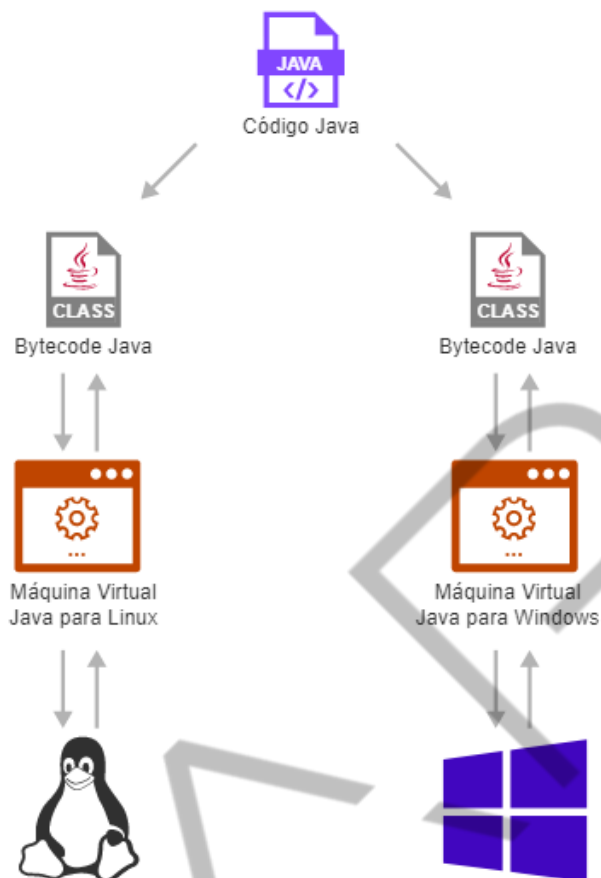


Figura 3 – Processo misto de compilação e interpretação
Fonte: DELGADO (2018)

O código Java passa inicialmente por uma compilação para uma linguagem intermediária (chamada de *bytecode*) que, posteriormente, é interpretada por uma máquina virtual (JVM – *Java Virtual Machine*) que realiza a tradução sob demanda na plataforma específica dessa JVM.

O motivo para essa adoção mista é permitir mesclar parcialmente o desempenho de uma aplicação compilada, com a interpretação sob demanda realizada na plataforma específica em que a aplicação será executada.

No caso do Java, existe mais um ponto forte nessa arquitetura, caso o desenvolvedor queira executar a aplicação em uma plataforma diferente, basta que seja instalada uma JVM específica para a sua execução, pois o *bytecode* é um código “universal”, necessitando apenas da tradução específica para cada plataforma.

Você deve estar pensando: e o Python, como ele atua? Podemos dizer que inicialmente ele é interpretado, pois a execução do código se dará sob demanda, característica das linguagens interpretadas, mas, dependendo da forma como o código Python é executado, em conjunto com o arquivo **.py** do código-fonte, um arquivo de nome **.pyc** é criado, que seria a sua versão compilada. O Python em determinadas situações cria o arquivo compilado para aumento de performance, mas, para execuções pontuais e corriqueiras, faz uso apenas da versão interpretada.

Ter o conhecimento sobre o uso de um compilador ou de um interpretador na linguagem de programação a ser analisada é importante no sentido de que, nas linguagens interpretadas, existe uma margem muito maior para a exploração de vulnerabilidades no que diz respeito à Programação Dinâmica, em comparação com as linguagens compiladas.

Antes disso, vamos trabalhar um pouco o conceito de Programação Dinâmica, que é o foco deste capítulo.

4 CONCEITO DE PROGRAMAÇÃO DINÂMICA

A Programação Dinâmica é uma técnica que permite a construção de comandos ou de *queries* (comandos para banco de dados) e a sua execução ao longo do ciclo de vida da aplicação.

Ou seja, essa técnica envolve a criação de código que não é possível saber *a priori* (antes da execução dele), o que costuma envolver informações fornecidas pelo usuário, por exemplo.

Pensemos em um exemplo prático de desenvolver um campo para pesquisa de nome de produto em um portal de e-commerce. Não é possível saber no momento do desenvolvimento do código quais são os produtos sobre os quais os consumidores irão pesquisar futuramente, no entanto, é factível pensar que, quando o usuário informa uma parte do nome do produto, realizamos uma *query* no banco de dados com base no nome informado.

Podemos pensar na *query* da seguinte forma:

```
public String exibir(String pesquisa, Connection conectar)
throws Exception {
    String saida = "";
    PreparedStatement estrutura = null;
    ResultSet resultado = null;

    estrutura = conectar.prepareStatement("SELECT * FROM
TB_PROD WHERE NM_PROD = ?");
    estrutura.setString(1, pesquisa);
    resultado = estrutura.executeQuery();

    while(resultado.next()) {
        saida += resultado.getString("NM_PROD") + " -
Salario: " + resultado.getDouble("VL_PROD") + "\n";
    }

    resultado.close();
    estrutura.close();
    return saida;
}
```

Código-fonte 1 – Exemplo de *query* dinâmica em Java
Fonte: Elaborado pelo autor (2018)

Em que o método **exibir** realiza uma pesquisa no banco de dados por produtos com o **NM_PROD** igual ao valor da variável *pesquisa*, que é informada pelo usuário.

Suponhamos que o usuário digite “Ventilador”, a *query* irá realizar uma pesquisa dentro da tabela **TB_PROD** por produtos com **NM_PROD** igual a “Ventilador”.

Essa forma de projetar código dá uma flexibilidade muito grande para o desenvolvedor, pois podemos executar código e *queries* com base nas informações que o usuário só nos fornecerá posteriormente.

Não é apenas para uso em *queries* de banco de dados que a programação dinâmica pode ser utilizada, há diversos usos e estes dependem muito da arquitetura da linguagem de programação.

Via de regra, as linguagens compiladas possuem menor margem para o uso da programação dinâmica (é necessário se utilizar de alguns “truques” para explorar vulnerabilidades), enquanto nas linguagens interpretadas, em virtude de o processo

de tradução ocorrer sob demanda, naturalmente suporta a geração de código conforme as informações recebidas do usuário.

Veremos mais algumas situações e exemplos nas linguagens Python e Java nas próximas seções, lembrando que não se trata de uma lista exaustiva, mas sim de uma ilustração das funcionalidades mais frequentemente utilizadas.

5 PROGRAMAÇÃO DINÂMICA EM PYTHON

O Python possui uma estrutura muito flexível em relação à execução de código, pois além de ser uma linguagem interpretada (como *default*, mas pode ser compilada em caso de necessidade), possui diversos mecanismos que permitem a geração dinâmica de código, assim como de alteração das características internas da linguagem, mecanismos estes que serão detalhados nos próximos subitens.

5.1 Método `eval()`

O método **`eval()`** nada mais é do que uma função que executa o código que é recebido como parâmetro (no formato texto) e que possui a seguinte sintaxe para execução:

```
eval(expression, globals=None, locals=None)
```

Código-fonte 2 – Sintaxe do método `eval()`
Fonte: Elaborado pelo autor (2018)

A finalidade para cada um dos parâmetros é a seguinte:

- *expression*: é o texto que contém o código a ser executado.
- *globals*: dicionário de variáveis globais que pode ser utilizado na execução da *expression*. Serve para limitar as funcionalidades globais a serem utilizadas. Em caso de nenhum dicionário ser usado, é utilizado o escopo atual.
- *locals*: dicionário de variáveis locais que pode ser utilizado na execução da *expression*. Serve para limitar as funcionalidades locais a serem utilizadas. Em caso de nenhum dicionário ser usado, é utilizado o escopo atual.

É mais prático apresentar um código de exemplo para mostrar o poder de execução do método **eval()**, como o código a seguir:

```
n = 20
print(eval('n * 2'))
```

Código-fonte 3 – Exemplo utilizando o método eval()
Fonte: Elaborado pelo autor (2018)

Repare que a expressão **n * 2** está entre aspas simples, ou seja, está sendo passada como texto para execução no **eval()**, assim como na expressão, é utilizada uma variável que foi declarada no código, o que permite o uso de variáveis que não necessariamente constam apenas na expressão (é possível utilizar variáveis da aplicação e do código dentro da expressão utilizada).

Acredito que o leitor já tenha pensado que é possível executar qualquer tipo de código ou comando no método **eval()**. Sim, é possível, e posteriormente será feita uma consideração sobre como conter o uso do **eval()** ou até mesmo evitar o seu uso.

Comandos nocivos podem ser utilizados quando o **eval()** não é dimensionado adequadamente:

```
print(eval(<código malicioso>))
```

Código-fonte 4 – Exemplo de código malicioso com o método eval()
Fonte: Elaborado pelo autor (2018)

Observação: não faça isso em casa ou em ambientes de terceiros!

O **<código malicioso>** se refere a códigos ou mesmo comandos *shell* que podem causar danos ou abrir vulnerabilidades no sistema. Mas como evitar isso?

Utilizamos os parâmetros *globals* e *locals* do **eval()** como medida inicial para não permitir acesso a qualquer informação da aplicação, isso será visto na parte prática deste capítulo.

5.2 Técnicas de metaprogramação em Python

5.2.1 O conceito de metaprogramação

Metaprogramação é uma forma de alterar o processo de criação de estruturas de uma linguagem de programação, e em linguagens como o Python é possível de se realizar para modificar a forma de criação de classes, métodos ou até mesmo tipos de variáveis.

A modificação pode ser feita antes de se criar uma nova estrutura ou mesmo durante a execução de uma aplicação.

Observação: metaprogramação é um conceito extremamente avançado, recomenda-se um estudo aprofundado antes de fazer uso dessas técnicas, ou mesmo evitar a sua utilização em aplicações de uso comercial por conta de efeitos colaterais que podem vir a aparecer durante a execução da aplicação.

Neste material, veremos um exemplo básico de metaprogramação que envolve criar um tipo de **classe** sem necessariamente criar um bloco de código que define essa classe da maneira tradicional.

E nesse novo tipo de **classe**, adicionar um método a ele dinamicamente (durante o tempo de execução do código).

Vejamos o código-fonte a seguir:

```
def bemvindo(self, nome):
    print("Seja bem vindo, " + nome)

Lista = type('Lista', (list,), dict(bemvindo=bemvindo))

nomes = Lista()
nomes.append("Antonio")
nomes.append("Beatriz")

print(nomes)
nomes.bemvindo("Rafael")
```

Código-fonte 5 – Exemplo básico de metaprogramação em Python
Fonte: Elaborado pelo autor (2018)

Esse código-fonte tem um método de nome **bemvindo()** que recebe um texto e o imprime na tela com o texto inicial **Seja bem vindo + <nome>**, que se trata de um método normal como qualquer outro em Python.

A metaprogramação acontece no momento em que o comando **type()** é utilizado, esse comando faz a criação de uma classe de nome **Lista**, que possui um tipo de **list** e um método de nome **bemvindo**. Repare que a criação da estrutura de uma classe foi feita dinamicamente, pois se poderia adicionar métodos e variáveis à vontade dentro do **dict** e, com isso, criar tipos de classe com base em blocos de código mais básicos.

Essa possibilidade de reaproveitamento de código de maneira dinâmica oferece uma flexibilidade muito grande para criar estruturas de dados e lógicas complexas conforme a necessidade, ao mesmo tempo, dá margem para a criação de *exploits*, quando usado em conjunto com técnicas de *code injection*.

Há maneiras não apenas de reescrever e criar classes, como também de alterar a estrutura de uma variável e sobrescrever métodos nativos do Python, inclusive alterando o comportamento usual da linguagem.

Agora vamos avaliar as possibilidades de programação dinâmica no Java, que possui uma arquitetura muito diferente.

6 PROGRAMAÇÃO DINÂMICA EM JAVA

O Java é uma linguagem de programação interpretada, logo, pode-se naturalmente pensar que a geração de código de forma dinâmica seria possível.

No entanto, o Java é uma das exceções a esse caso, em boa parte devido à proposta da linguagem no momento em que foi criada, que foi a de atender o desenvolvimento de sistemas em geral e de forma ubíqua, portanto, o uso de programação dinâmica é bem limitado devido a considerações de segurança, especialmente em relação ao uso de metaprogramação.

Mesmo que com várias limitações, é possível utilizar programação dinâmica em duas situações bem específicas, a serem vistas mais detalhadamente nas próximas subseções.

6.1 Geração de queries

Um exemplo bem similar ao do Código-fonte “Exemplo de *query* dinâmica em Java” é apresentado a seguir, mas utilizando-se de uma variedade maior de parâmetros:

```
public void gravar(String nome, double valor, int quantidade,
Connection conectar) throws Exception {
    PreparedStatement estrutura = null;
    estrutura = conectar.prepareStatement("INSERT INTO
TB_PROD VALUES (?, ?, ?)");
    estrutura.setString(1, nome);
    estrutura.setDouble(2, valor);
    estrutura.setInt(3, quantidade);
    estrutura.executeUpdate();
    estrutura.close();
}
```

Código-fonte 6 – Exemplo de *query* dinâmica em Java com três parâmetros
Fonte: Elaborado pelo autor (2018)

O código basicamente realiza uma operação de **insert** em um banco de dados, criando um registro com três campos: *nome*, *valor* e *quantidade*.

Nesse caso, é possível observar que se usa indiretamente a programação dinâmica, apesar de o código Java não ser gerado dinamicamente, é possível gerar *queries* SQL no formato texto e que serão enviadas para o banco de dados posteriormente.

Existem outras situações em que se usa indiretamente programação dinâmica, uma delas refere-se à geração de código HTML por meio de um arquivo JSP ou um Servlet para exibição em browser, em que o texto que contém o HTML é criado à parte e depois enviado para renderização.

Uma dica é procurar as integrações de um sistema e verificar se usam linguagens distintas, pois muito provavelmente o código que faz a integração pode conter processamento de código por meio de variáveis contendo texto, o que pode se tornar uma vulnerabilidade.

6.2 Técnicas de metaprogramação em Java

O Java basicamente não suporta metaprogramação nativamente, ao menos não suporta no mesmo nível que o Python permite.

Porém, é possível obter algumas informações sobre a estrutura de uma classe ou de um objeto durante a execução da aplicação, o que pode ser útil para diagnosticar eventuais vulnerabilidades e pontos de entrada dentro do código.

Utilizando a Java Reflection API, é possível inspecionar métodos contidos em uma determinada classe por meio dos seguintes métodos:

- **<nome da classe>.getMethods()**: lista todos os métodos da classe, inclusive métodos herdados.
- **<nome da classe>.getDeclaredMethods()**: lista apenas os métodos declarados na classe.
- **<nome da classe>.getReturnType()**: lista o tipo de retorno para cada método.

A seguir, um exemplo mostrando a listagem dos métodos de uma classe:

```
import java.lang.reflect.Method;

public class Teste {
    private String nome;

    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public static void main(String[] args) {
        Teste classe = new Teste();
        Method[]          metodos          =
        classe.getClass().getDeclaredMethods();

        for (Method metodo: metodos) {
            System.out.println(metodo.getName());
        }
    }
}
```

Código-fonte 7 – Exemplo de código Java que lista os métodos de uma classe

Fonte: Elaborado pelo autor (2018)

Esse código basicamente cria um objeto do tipo **Teste**, obtém-se a listagem dos métodos contidos nele (por meio do método **getDeclaredMethods**), que retorna uma lista do tipo **Method**.

Depois um loop **for** é feito para obter o nome de cada método (pelo método **getName**) e o exibe na tela.

O resultado na tela é a exibição dos nomes **main**, **getNome** e **setNome**, que são os métodos contidos na classe.

Na linguagem Java, felizmente (ou infelizmente) não é possível utilizar metaprogramação de forma direta, mas consultar as características de uma classe é viabilizado pela Java Reflection API.

Agora, vamos juntar todo esse conhecimento e fazer a análise de vulnerabilidade em um código mais complexo, no laboratório a seguir.

7 LABORATÓRIO – ANÁLISE DE VULNERABILIDADES

O *Smart Meter* é um dos dispositivos encontrados em prédios comerciais ou mesmo em residências para mensuração e coleta de informações, como consumo de energia elétrica e/ou de água, por exemplo.

Dentro do conceito de *Smart City*, o *Smart Meter* permite a agregação e posterior envio das informações para uma central que processará as informações em tempo real.

Portanto, um *Smart Meter* possui ou está integrado a uma malha variada de sensores, cujos dados serão armazenados temporariamente nele e depois serão enviados para a camada de *analytics*.

Considere que temos acesso ao código-fonte de um *Smart Meter* simples, que recebe as informações de diversos sensores (e deve ter flexibilidade para trabalhar os dados nos mais variados formatos) e que foi desenvolvido ao longo do tempo por diversos programadores.

E agora cabe a você analisar esse código e fazer melhorias pontuais nele de forma que esse *Smart Meter*, quando instalado (e exposto a uma rede pública), execute o código da maneira correta e cumpra o seu objetivo.

```
import os

def calcEnergy(kwh, price):
    return kwh * price

def calcWater(m3, price):
    return m3 * price

metodo = raw_input("Selecione a conta utilizada: ")

if(metodo == 'calcEnergy'):
    kwh = input("Digite o consumo utilizado: ")
    price = input("Digite o preco da energia: ")
    print("A fatura de energia foi de R$ " + str(eval(metodo
+ "(%s, %s)" % (kwh, price))))
elif (metodo == 'calcWater'):
    m3 = input("Digite o consumo utilizado: ")
    price = input("Digite o preco da agua: ")
    print("A fatura de agua foi de R$ " + str(eval(metodo +
"(%s, %s)" % (m3, price))))
else:
    m3 = 0
    kwh = 0
    price = 0
    print(eval(metodo))
```

Código-fonte 8 – Código Python do Smart Meter
Fonte: Elaborado pelo autor (2018)

O código lê primeiro qual a conta (método) a ser utilizada, se for **calcEnergy**, pede o consumo em Kwh e depois o preço da energia para a conta ser realizada. Caso seja **calcWater**, pede o consumo em metros cúbicos e depois o preço da água e faz a conta.

Se nenhum dos dois métodos for digitado, o código executa o texto digitado pelo usuário.

A primeira etapa de análise é testar o código para as entradas consideradas válidas e verificar se o resultado está correto, então começamos pelo método **calcEnergy**, passando depois para o método **calcWater**:

```
Selecione a conta utilizada: calcEnergy
Digite o consumo utilizado: 250
Digite o preco da energia: 1.50
A fatura de energia foi de R$ 375.0
```

Figura 4 – Teste do método calcEnergy
Fonte: Elaborado pelo autor (2018)

```
Selecione a conta utilizada: calcWater
Digite o consumo utilizado: 100
Digite o preco da agua: 2.50
A fatura de agua foi de R$ 250.0
```

Figura 5 – Teste do método calcWater
Fonte: Elaborado pelo autor (2018)

A execução dos testes utilizando valores válidos mostrou que o código “funciona” para entradas de dados bem-comportadas. Mas o que aconteceria no caso de digitarmos um comando no lugar da conta?

```
Selecione a conta utilizada: os.listdir("/")
['$GetCurrent', '$Recycle.Bin', '$SysReset', 'Arquivos de Programas',
Process finished with exit code 0
```

Figura 6 – Teste injetando comando
Fonte: Elaborado pelo autor (2018)

O comando `os.listdir("/")` exibe a lista de arquivos da raiz do drive C da máquina, e esses tipos de comando não deveriam ser permitidos nessa aplicação.

Existem duas maneiras de resolver esse problema:

- Reduzir o escopo de atuação do `eval()`.
- Remover as chamadas `eval()`.

7.1 Redução de escopo do eval()

É possível limitar o acesso a variáveis ou métodos de que o **eval()** se utiliza, isso envolve colocar valores para os parâmetros *globals* e *locals*.

- *Globals* define a que variáveis globais o **eval()** terá acesso, o que limita bastante o problema de roubo de valores de variáveis.
- E *locals* define a que variáveis locais o **eval()** terá acesso.

No nosso caso, como não temos variáveis globais sendo utilizadas, basta configurar o *locals* com as variáveis e os métodos que utilizaremos no **eval()**, então o código ficaria assim:

```
import os

def calcEnergy(kwh, price):
    return kwh * price

def calcWater(m3, price):
    return m3 * price

metodo = raw_input("Selecione a conta utilizada: ")

if(metodo == 'calcEnergy'):
    kwh = input("Digite o consumo utilizado: ")
    price = input("Digite o preco da energia: ")
    print("A fatura de energia foi de R$ " + str(eval(metodo + " (%s, %s)" % (kwh, price))))
elif (metodo == 'calcWater'):
    m3 = input("Digite o consumo utilizado: ")
    price = input("Digite o preco da agua: ")
    print("A fatura de agua foi de R$ " + str(eval(metodo + " (%s, %s)" % (m3, price))))
else:
    m3 = 0
    kwh = 0
    price = 0
    print(eval(metodo, {'m3':m3, 'kwh':kwh, 'price':price, 'calcEnergy':calcEnergy, 'calcWater':calcWater}))
```

Código-fonte 9 – Código Python com *locals* no Smart Meter
Fonte: Elaborado pelo autor (2018)

Repare que o **eval()** do **else** está com um **dict** listando apenas as variáveis e os métodos que podem ser acessados.

Também seria útil colocar esse **dict** nos demais **eval()**, a saber.

Agora vamos testar o mesmo comando com essa blindagem de escopo reduzido para o **eval()**:

```
Selecione a conta utilizada: os.listdir("/")
Traceback (most recent call last):
  File "C:/Users/rafae/PycharmProjects/cr/code2a.py", line 23, in <module>
    print(eval(metodo, {'m3':m3, 'kwh':kwh, 'price':price, 'calcEnergy':calc
  File "<string>", line 1, in <module>
NameError: name 'os' is not defined
```

Figura 7 – Eval() com escopo reduzido
Fonte: Elaborado pelo autor (2018)

Note que o método **listdir()** sequer conseguiu ser acessado, pois não deixamos o módulo **os** disponibilizado para execução no **eval()**. Essa é uma maneira rápida de resolver as questões envolvendo o **eval()** quando não é possível (ou nem é desejável) refatorar o código.

7.2 Remoção do eval()

A remoção do **eval()** seria a opção mais correta tecnicamente falando, mas fazer isso envolve ter um conhecimento aprofundado sobre como o código está se comportando, assim como testar bastante o código refatorado.

No nosso caso, realizar isso é bem fácil, basta trocar a chamada do **eval()** pela chamada direta do método, assim como trocar o **eval()** do **else** por uma mensagem de erro.

Fazendo essas alterações, o código fica assim:

```
import os

def calcEnergy(kwh, price):
    return kwh * price

def calcWater(m3, price):
    return m3 * price

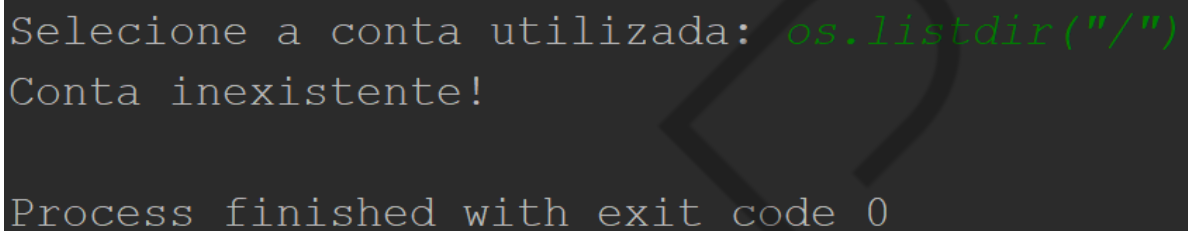
metodo = raw_input("Selecione a conta utilizada: ")

if(metodo == 'calcEnergy'):
    kwh = input("Digite o consumo utilizado: ")
    price = input("Digite o preco da energia: ")
    print("A fatura de energia foi de R$ " +
    str(calcEnergy(kwh, price)))
elif (metodo == 'calcWater'):
```

```
m3 = input("Digite o consumo utilizado: ")
price = input("Digite o preco da agua: ")
print("A fatura de agua foi de R$ " + str(calcWater(m3,
price)))
else:
    print("Conta inexistente!")
```

Código-fonte 10 – Código Python sem eval() no *Smart Meter*
Fonte: Elaborado pelo autor (2018)

Agora vamos testar com o comando:



```
Selecione a conta utilizada: os.listdir("/")
Conta inexistente!

Process finished with exit code 0
```

Figura 8 – Sem eval() no código
Fonte: Elaborado pelo autor (2018)

A chamada **eval()** genérica foi substituída por uma mensagem de erro. Se testarmos os métodos **calcEnergy** e **calcWater**, veremos que o comportamento dos mesmos não foi alterado, então o processo de refatoração foi bem-sucedido.

Caso queria explorar mais o funcionamento do **eval()**, existe uma documentação bem detalhada do Python a respeito: <<https://docs.python.org/2/library/functions.html#eval>> (em inglês).

CONCLUSÃO

Programação dinâmica oferece muita flexibilidade para o desenvolvedor gerar código sob demanda e conforme as informações fornecidas pelo usuário, mas como toda funcionalidade flexível, é necessário saber onde utilizar e como utilizar.

Neste capítulo, vimos que nem todas as linguagens de programação permitem o uso extensivo de técnicas de programação dinâmica, assim como algumas técnicas como o uso do **eval()** no Python ou de geração de *queries* no Java.

Por fim, uma situação prática com código utilizando **eval()** foi apresentada para que uma análise de vulnerabilidade fosse conduzida, em conjunto com várias considerações a respeito do uso do **eval()** em aplicações.

Mas o que aconteceria se, no exemplo prático, não pudéssemos retirar o **eval()** por algum motivo? Será que apenas limitando o uso de *local/s* resolveria todas as vulnerabilidades de código? A resposta é não... infelizmente.

EMANIP

REFERÊNCIAS

DELGADO, Humberto. **Material da Disciplina Domain Driven Design. Aula 03 – Tipos de Linguagem.**

EMEND